

CAPSTONE PROJECT DOCUMENTATION

vsensor-hub

A Virtual Interrupt-Driven Sensor Device:
Linux Kernel Driver and C++ User-Space Stack

Student Name	[Your Name]
Roll / Registration No.	[Your Roll No.]
Course	[Course Name]
Instructor	[Instructor Name]
Date	[Submission Date]
Repository	https://github.com/[username]/vsensor-hub

Languages: C (kernel module), C++17 (user space) | Platform: Linux

Table of Contents

1. Abstract
2. Introduction and Objectives
3. Compliance with Capstone Requirements
4. System Architecture
5. Kernel Driver Design
6. Device Interface Specification
7. Synchronization Design
8. User-Space Components
9. Repository Structure
10. Build, Load and Run
11. Testing
12. Results
13. Limitations and Future Work
14. Conclusion
15. References

1. Abstract

This project implements a *virtual sensor device* as a Linux character device driver written in C, together with a layered C++ software stack that uses it. Because no physical hardware is required, the device is simulated: a high-resolution kernel timer plays the role of a hardware interrupt, periodically generating temperature samples that are stored in a kernel ring buffer. User programs retrieve the samples through the standard file interface (`/dev/vsensor0`) using blocking and non-blocking reads, `poll()`, and `ioctl()` control commands. A C++ library, a command-line monitor and a multithreaded logger daemon complete the system. The project demonstrates core device-driver concepts and hardware/software architecture concepts: the user/kernel boundary, interrupt handling, producer-consumer buffering, register-style configuration, and synchronization in atomic and process contexts.

2. Introduction and Objectives

Device drivers are the layer through which an operating system communicates with hardware. Understanding how a driver receives asynchronous events, buffers data, synchronizes with user programs and exposes a controlled interface is central to systems and computer-architecture education. Since physical hardware is not always available, this project simulates a periodic sensor so that the same driver mechanisms can be implemented, tested and demonstrated on any Linux machine.

Objectives

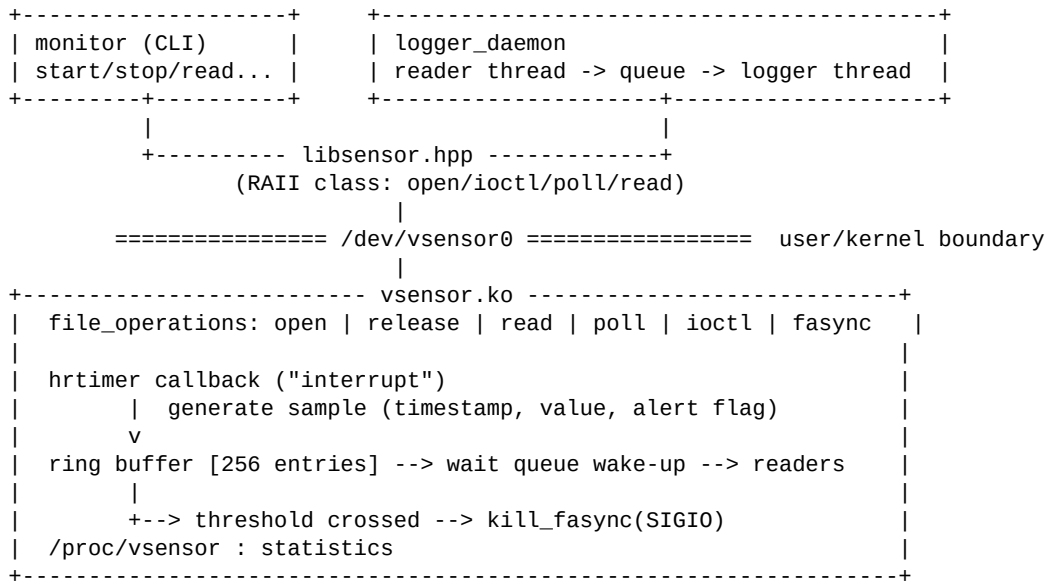
- Implement a Linux character device driver in C using the standard kernel APIs.
- Simulate an interrupt-driven hardware source and handle it in atomic context.
- Provide data to user space through blocking, non-blocking and polled I/O.
- Expose a register-like control interface through `ioctl` and a statistics interface through `/proc`.
- Apply correct synchronization (spinlock and mutex) and document the reasoning.
- Build a layered C++ user-space stack: library, CLI and multithreaded daemon.
- Publish a well-structured, documented project on GitHub.

3. Compliance with Capstone Requirements

Requirement	How this project satisfies it
1. C/C++ only	Kernel module in C; library, CLI and daemon in C++17. No other language is used (build scripts are shell and Makefiles only).
2. Linux OS only	Uses <code>kbuild</code> , kernel headers, <code>insmod/rmmod</code> ; developed and run on Ubuntu Linux.
3. Linux device driver concepts	Character device registration (<code>alloc_chrdev_region</code> , <code>cdev</code> , <code>class_create</code> , <code>device_create</code>), <code>file_operations</code> , <code>ioctl</code> , <code>copy_to_user</code> , wait queues, <code>poll</code> , <code>fasync/SIGIO</code> , <code>hrtimer</code> , <code>spinlock</code> , <code>mutex</code> , <code>procfs</code> , module parameters.
4. Software / hardware architecture	Simulated register map, interrupt source, ring buffer (FIFO), layered design across the user/kernel boundary, producer-consumer threading.
5. GitHub submission	Structured repository with source, <code>README.md</code> , documentation, scripts and tests (Section 9).

4. System Architecture

The system has three layers. The kernel layer contains the driver and the simulated hardware. The library layer wraps the system-call interface in a safe C++ class. The application layer contains the user-facing programs.



Data flow

- The hrtimer expires periodically (default 100 ms) and its callback runs in atomic context.
- The callback creates a sample, takes the spinlock, and pushes the sample into the ring buffer; if the buffer is full the sample is dropped and counted.
- The callback wakes sleeping readers and, if the value exceeds the threshold, signals processes registered for asynchronous notification.
- A user process sleeps in `read()` or `poll()` until data is available, then receives whole samples.

5. Kernel Driver Design

5.1 Device registration

At module load, the driver allocates a device number (`alloc_chrdev_region`), initializes and adds a cdev, creates a device class and node (so `/dev/vsensor0` appears automatically through udev), and creates `/proc/vsensor`. Error paths unwind in reverse order. The module also supports kernel-version differences (for example `class_create` and `hrtimer_setup` signature changes) using `LINUX_VERSION_CODE` guards.

5.2 Simulated hardware

The sensor is a software model. A pseudo-random generator (`xorshift`) produces temperature values between 15.0 and 35.0 degrees C, stored in milli-degrees as a 32-bit signed integer. Each sample carries a nanosecond timestamp and an alert flag.

5.3 Ring buffer

A fixed array of 256 samples is managed with *head* (producer index) and *tail* (consumer index). One slot is kept empty to distinguish full from empty, giving 255 usable entries. The buffer is empty when head equals

tail. When full, the newest sample is dropped and the *dropped* counter is incremented, so overflow is visible rather than silent.

5.4 Blocking, non-blocking and polled reads

`read()` returns an integral number of samples. If the buffer is empty it sleeps on a wait queue (`wait_event_interruptible`), returns `-EAGAIN` if the file was opened with `O_NONBLOCK`, and returns `-ERESTARTSYS` if interrupted by a signal. `poll()` registers the wait queue and reports readability when samples are present, enabling `select/poll/epoll` use.

5.5 Asynchronous notification

A process may enable `O_ASYNC` on the descriptor. When a sample exceeds the configured threshold, the driver calls `kill_fasync` and the process receives `SIGIO`, modelling a threshold interrupt line.

6. Device Interface Specification

6.1 Shared data structures (kernel/vsensor.h)

Structure	Fields	Description
<code>struct vs_sample</code>	<code>ts_ns</code> (u64), <code>value</code> (s32), <code>alert</code> (u32)	One sample: monotonic timestamp in ns, temperature in milli-degrees C, 1 if above threshold
<code>struct vs_stats</code>	<code>produced</code> , <code>dropped</code> , <code>consumed</code> , <code>alerts</code> (all u64)	Cumulative counters, read atomically under the spinlock

6.2 Control commands (ioctl, magic 'v')

Command	Argument	Behaviour
<code>VS_IOC_START</code>	none	Start the hrtimer (idempotent)
<code>VS_IOC_STOP</code>	none	Cancel the timer
<code>VS_IOC_SET_RATE</code>	u32 period in ms (1-60000)	Set sampling period; takes effect at the next start. Out-of-range returns <code>-EINVAL</code>
<code>VS_IOC_SET_THRESHOLD</code>	s32 milli-degrees C	Set alert threshold (default 30000 = 30.0 C)
<code>VS_IOC_GET_STATS</code>	<code>struct vs_stats</code> (out)	Copy counters to user space
<code>VS_IOC_CLEAR</code>	none	Discard buffered samples

Unknown commands return `-ENOTTY`. User pointers are accessed only through `get_user` and `copy_to_user`, and faults return `-EFAULT`.

6.3 Logical register map

Logical register	Interface	Meaning
<code>CTRL</code>	<code>START / STOP ioctl</code>	Enable or disable the sensor clock
<code>SAMPLE_RATE</code>	<code>SET_RATE ioctl</code> , module parameter <code>period_ms</code>	Sampling period

Logical register	Interface	Meaning
THRESHOLD	SET_THRESH ioctl	Alert level
STATUS	GET_STATS ioctl, /proc/vsensor	Counters and current configuration
DATA FIFO	read() on /dev/vsensor0	Stream of struct vs_sample

6.4 procfs output (/proc/vsensor)

Reports: running state, period (ms), threshold, and the produced, consumed, dropped and alert counters.

7. Synchronization Design

Resource	Mechanism	Reason
Ring buffer indices, statistics, threshold	Spinlock with irqsave (spin_lock_irqsave)	Accessed from the timer callback, which runs in atomic context and cannot sleep, and from process context. A mutex would be illegal in the callback; disabling local interrupts prevents deadlock if the timer fires while a process holds the lock.
Configuration changes (ioctl)	Mutex (cfg_lock)	Process context only; serializes start/stop/rate changes. Safe to hold while calling functions that may sleep.
Waiting for data	Wait queue	Readers sleep instead of busy-waiting; the callback wakes them with wake_up_interruptible.
Stopping the timer	hrtimer_cancel under the mutex only	hrtimer_cancel may wait for a running callback, so it must never be called while holding the spinlock the callback needs.

The copy to user space happens *after* the spinlock is released, because copy_to_user can fault and sleep. The sample is first copied to a local variable under the lock, then to the user buffer.

8. User-Space Components

8.1 libsensor.hpp

A header-only C++17 class Sensor using RAI: the constructor opens the device and throws std::runtime_error on failure; the destructor closes it; copying is disabled. Methods wrap each ioctl (start, stop, clear, setRateMs, setThreshold, stats), and read(max, timeout_ms) combines poll() with read() to return a vector of samples or an empty vector on timeout.

8.2 monitor (command-line tool)

Provides the commands start, stop, clear, rate, thresh, stats, read N and watch. It converts raw samples into readable output (timestamp in seconds, temperature in degrees C, ALERT marker).

8.3 logger_daemon

Demonstrates the producer-consumer pattern with two threads. The *reader thread* polls the device and appends samples to a shared queue; the *logger thread* waits on a condition variable and writes batches to a CSV file (timestamp_s, temp_c, alert). Separating the threads prevents slow disk I/O from delaying reads and overflowing the kernel buffer. SIGINT/SIGTERM trigger a clean shutdown that drains the queue, stops the sensor and prints final statistics.

9. Repository Structure

```
vsensor-hub/
  README.md      Makefile      .gitignore
  kernel/        vsensor.c    vsensor.h    Makefile
  lib/           libsensord.hpp
  app/           monitor.cpp  logger_daemon.cpp
  scripts/       load_module.sh  unload_module.sh
  tests/         run_tests.sh
  docs/          architecture.md  register_map.md
```

10. Build, Load and Run

Prerequisites: Linux with kernel headers, gcc, g++, make (Ubuntu: `sudo apt install build-essential linux-headers-$(uname -r)`). On Windows hosts, run Ubuntu in a virtual machine.

```
make # build module and user programs
./scripts/load_module.sh # insmod, set permissions on /dev/vsensor0
./monitor start
./monitor read 10
cat /proc/vsensor
./logger_daemon out.csv # Ctrl+C to stop
./scripts/unload_module.sh
```

11. Testing

Test	Method	Expected result
Functional read	Start sensor at 10 ms, read 20 samples with <code>./monitor read 20</code>	Exactly 20 sample lines printed
Statistics	<code>./monitor stats</code> and <code>/proc/vsensor</code>	Counters present and consistent (produced $\geq$ consumed)
Overflow handling	Stop, clear, set 1 ms, start, wait 3 s with no reader	dropped counter greater than zero; no crash
Threshold alert	Set a low threshold and read samples	Samples marked ALERT; alerts counter increases
Blocking read	Read with the sensor stopped, then start it from another terminal	Reader sleeps, then wakes when data arrives
Logger daemon	Run <code>./logger_daemon out.csv</code> for 5 s, Ctrl+C	CSV written; final statistics printed; clean exit
Module lifecycle	<code>insmod</code> , <code>rmmmod</code> , check <code>dmesg</code>	No errors or leaks reported

Tests 1-3 are automated in `tests/run_tests.sh`; the others are manual checks.

12. Results

Insert here the actual output captured on your system. Suggested content:

- Output of `./monitor read 10`
- Output of `./tests/run_tests.sh` (ending with ALL TESTS PASSED)
- Contents of `/proc/vsensor`
- First lines of the CSV file produced by `logger_daemon`
- Relevant `dmesg` lines showing module load and unload
- Your kernel version (`uname -r`) and Linux distribution

[Paste screenshots / terminal output here]

13. Limitations and Future Work

- Only one device instance is supported and the data is simulated.
- The sampling rate change is applied at the next start rather than immediately.
- A single reader is assumed; multiple concurrent readers share one consumer index.
- Future work: mmap of the ring buffer with a read-versus-mmap latency benchmark; moving processing to a tasklet or workqueue (bottom half); a platform-driver and device-tree binding; per-open-file buffers; a sysfs attribute interface.

14. Conclusion

The project delivers a complete device-driver stack on Linux: a simulated interrupt source, a buffered character device with blocking, polled and asynchronous access, an `ioctl` control interface, `procfs` statistics, and a layered C++ user-space system. It demonstrates the principal responsibilities of a driver (handling asynchronous events, safely moving data across the user/kernel boundary, and synchronizing between atomic and process contexts) while remaining runnable on any Linux machine without special hardware.

15. References

- Corbet, Rubini, Kroah-Hartman. *Linux Device Drivers*, 3rd ed., O'Reilly.
- The Linux Kernel documentation: <https://docs.kernel.org> (driver API, hrtimers, locking).
- Love, R. *Linux Kernel Development*, 3rd ed.
- Linux man pages: `poll(2)`, `ioctl(2)`, `fcntl(2)`.
- Stroustrup, B. *The C++ Programming Language* (RAII, threading).